

MAST32001 Computational Statistics

Course Assignment

Klaus Nordhausen

WS 2026

General instructions

The assignment is completed in the groups formed for the course. Each group submits one joint final report covering both tasks. The report should demonstrate statistical understanding, sound computational methodology, reproducibility, and careful interpretation of the results.

All group members take **full responsibility for the complete submitted work**, not only for the sections that they personally prepared. Every group member is therefore expected to understand the methods, code, results, and conclusions presented in the final report.

Report and presentation

The final report must be written in clear, formal academic language. Informal or conversational language should be avoided. The report should be written as a coherent scientific document rather than as a collection of answers to individual questions.

The report must clearly state the **names of all group members**. The final report may be at most **25 pages**, including figures, tables, references, and appendices. This is a strict maximum; groups should therefore select and present the essential methodology, results, diagnostics, and discussion rather than including all intermediate output.

The final PDF should be named `MAST32001_Group_X.pdf`, where `X` is the group number.

The main text should primarily consist of connected prose. Bullet-point lists should generally be avoided except where they are clearly the most appropriate way to present information. Mathematical notation should be defined when introduced, and methodological choices should be explained and justified.

All figures must have informative axis labels and a descriptive caption. All tables must have a descriptive caption and clearly labelled columns and, where appropriate, rows. Every figure and table included in the report must be explicitly referred to and discussed in the main text. Figures and tables should support the argument of the report; unexplained output should not be included.

Raw R output, long code listings, or collections of diagnostic plots should not replace written interpretation. The report should explain what the computational results show and why they matter for the statistical problem.

Report structure

For each of the two tasks, the report should broadly follow the structure indicated in the corresponding point allocation at the end of this assignment. The point allocation should be understood not only as a grading rubric but also as guidance on the expected emphasis and organization of the report.

Appropriate section and subsection headings should be used. The report should nevertheless read as a coherent scientific document rather than as a sequence of separate answers to individual assignment questions.

GitHub and reproducibility

All work on the assignment must be carried out in the group's GitHub repository. The **TA assigned to the group must be invited to the repository** and given access throughout the assignment period.

The repository should contain all code and other material required to reproduce the analyses and results in the report. Code must be appropriately organized and commented so that its purpose and the main computational steps can be understood by another reader.

Git commits must be meaningful and well documented. Commit messages should describe the work performed; vague commit messages such as “changes”, “update”, or “final” should be avoided. The Git history should provide a reasonable record of the development of the group project.

Each group member is expected to contribute to the GitHub repository using their own account. The commit history should reflect the development of the project and the contributions of the group members. The **URL of the GitHub repository must be included in the final report**.

Group work and individual responsibility

The assignment is a group submission, but participation is also assessed individually. Each student must submit a **self and peer assessment in Moodle**.

In the self and peer assessment, each student should evaluate their own performance and contribution to the project and the performance and contribution of **each of the other group members**. The assessment should reflect the actual division of work, participation in the project, and contribution to the final result.

The group is jointly responsible for the final submission. Division of labour within the group does not remove responsibility for other parts of the work: **every group member takes full responsibility for the complete report, analysis, code, and submitted results**.

Generative AI

The use of **generative AI is not allowed for any part of this assignment**. This includes, but is not limited to, generating or rewriting report text, generating or modifying code, solving

theoretical derivations, producing interpretations, or generating figures or analyses.

The submitted report, code, derivations, analyses, and interpretations must be the work of the group members.

Submission and deadline

The deadline for the complete assignment is

25 October 2026 at 20:00

By this deadline:

1. the **final group report** must be submitted by email to both the **course teacher** and the **TA assigned to the group**;
2. **each student** must have submitted their self and peer assessment in Moodle; and
3. the GitHub repository must contain the final version of all code and material used for the submission.

The submission email must state the **group number and the names of all group members** and must include the final PDF report as an attachment. It must also include the **URL of the group's GitHub repository**. If the group requests an early oral examination, this must be stated clearly in the submission email.

At **20:00 on 25 October 2026, the GitHub repository is considered frozen**. No files, code, documentation, results, or other material in the repository should be changed after the deadline.

Groups should therefore make sure well before the deadline that the final report has been sent to both recipients, all required individual Moodle submissions have been made, and the GitHub repository contains the final reproducible version of the project.

Oral examination

After submitting the assignment, each group must book one of the available **oral examination slots**. The examination slots will be made available later in Moodle.

All group members must attend the oral examination together. The oral examination is part of the assignment process, and students should be prepared to discuss the complete report, methodology, code, results, and conclusions. As stated above, every group member is responsible for the complete submitted work.

Groups that submit the **complete assignment by 18 October 2026** may also be offered oral examination times before the final assignment deadline of 25 October 2026. If a group would like an earlier oral examination, this must be **clearly stated in the email when the final report is submitted**.

If an earlier oral examination is important for the group, contacting the course teacher by email **in advance of the submission** is strongly encouraged so that the possibility of an earlier examination time can be discussed.

An early oral examination is possible only after the complete assignment has been submitted. Once the assignment has been submitted for an early oral examination, the submitted report and the corresponding GitHub version should be regarded as final.

Task 1: Time series bootstrap and the ladle estimator

The purpose of this task is to study bootstrap methodology for dependent observations and to use it in a dimension-estimation problem for multivariate time series.

The task is based on the following paper, which should be used as the main reference for the ladle estimator in this task:

Nordhausen, K. and Virta, J. (2018). *Ladle Estimator for Time Series Signal Dimension*. In *2018 IEEE Statistical Signal Processing Workshop (SSP)*, pp. 428–432. DOI: 10.1109/SSP.2018.8450695.

Let $\mathbf{x}_1, \dots, \mathbf{x}_n$ be a p -variate time series. The goal is to estimate the dimension of the signal subspace using the ladle estimator and to investigate how bootstrap methodology must be adapted when observations are serially dependent.

Part A: Whitening and autocovariance matrices

Implement the preprocessing and matrix calculations needed for the time-series version of the ladle estimator.

Your implementation should include whitening of the observed time series and estimation of the relevant autocovariance matrices. Clearly state the conventions you use for centering, scaling, lagging, and normalization.

Explain why whitening is used in this setting and verify numerically that your implementation has the expected effect on the sample covariance matrix.

The main analysis must use your own implementation of these computational steps rather than a ready-made implementation of the complete ladle procedure.

When more than two matrices need to be jointly diagonalized, you are **not required to implement a joint-diagonalization algorithm yourself**. For this step, you should use the function `rjd()` from the R package JADE. For the ordinary two-matrix/eigenvalue case, standard R matrix operations may be used. The remaining computational steps specified in this task should be implemented as described in the assignment.

Part B: Bootstrap for dependent observations

In the course we discussed block-bootstrap ideas for dependent observations. The **stationary bootstrap was not covered in detail in the lectures** and is deliberately included here as a small piece of independent study.

For the stationary bootstrap, study the original paper

Politis, D. N. and Romano, J. P. (1994). *The Stationary Bootstrap*. Journal of the American Statistical Association, **89**(428), 1303–1313. DOI: 10.1080/01621459.1994.10476870.

In particular, you should understand the resampling construction in the paper well enough to explain the method in your own words and implement it yourself. You are not expected to reproduce the theoretical consistency or weak-convergence arguments from the paper.

Implement the following two bootstrap methods for multivariate time-series observations:

1. the **fixed-block bootstrap**;
2. the **stationary bootstrap of Politis and Romano (1994)**.

Explain how each method generates a bootstrap sample and identify the tuning parameter controlling the block lengths. For the stationary bootstrap, explain in particular how the random block lengths are generated, how their expected length is determined by the method's parameter, and how the construction differs from using blocks of a fixed length.

Discuss why ordinary iid resampling of individual observations is generally inappropriate for a dependent time series and why block-based resampling is used instead.

Your functions should work for a general multivariate data matrix and should make it possible to generate the bootstrap samples needed by the ladle estimator. The stationary-bootstrap implementation must be based on your understanding of Politis and Romano (1994), rather than on calling a ready-made stationary-bootstrap function.

Investigate how the choice of block length, or expected block length for the stationary bootstrap, affects the resampled series. Use suitable graphical or numerical summaries to demonstrate how the bootstrap methods preserve local dependence differently from iid resampling.

Part C: The ladle quantities

Implement the quantities f , g , and ϕ used in the ladle estimator as defined in the paper.

Your report should explain what information is represented by the eigenvalue-based part and by the bootstrap-variability part of the criterion, and how these quantities are combined to estimate the dimension.

Check your implementation carefully on simulated data. Intermediate computational checks should be described where they help establish that the implementation is correct.

Part D: Simulation study

Use the simulation setting specified as **Setting 1** in the paper to investigate the behaviour of your implementation.

You are not required to reproduce every numerical result or figure in the paper. Instead, reproduce the **main qualitative behaviour** of the estimator under Setting 1 and use the simulation to demonstrate that your implementation works as intended.

Compare the fixed-block and stationary bootstrap approaches. Choose reasonable block-length settings and justify the choices. Investigate whether the estimated dimension is sensitive to the bootstrap method, block length, or sample size.

Your simulation study should contain enough independent repetitions to support the conclusions, but computational effort should be used sensibly. Report Monte Carlo results in a way that makes the uncertainty and variability of the simulation visible.

Part E: Application to the course data

Apply the methods to the time-series data set provided for the assignment in Moodle.

Use both the fixed-block and stationary bootstrap versions of the ladle estimator. Select and justify the block-length choices used in the final analysis.

Your analysis should include:

- the estimated signal dimension and the evidence supporting it;
- an appropriate visualization of the ladle criterion and its components;
- a comparison of the results obtained with the two bootstrap methods;
- a sensitivity analysis for the block-length choice;
- a discussion of what the results imply for the structure of the observed multivariate time series.

If the two bootstrap approaches or different reasonable block lengths lead to different conclusions, this should be investigated and discussed rather than hidden by reporting only one result.

Reporting requirements

The report should contain a concise description of the time-series bootstrap problem, the implemented methodology, the simulation study, and the real-data analysis. Mathematical definitions from the paper should be explained in your own presentation rather than copied without context.

The choices of bootstrap method, block length, simulation settings, and computational parameters must be justified. Figures and tables should be used to support the statistical argument, and computational output should be interpreted rather than merely displayed.

Ready-made software may be used to check selected calculations, but the whitening, auto-covariance calculations, bootstrap procedures, and ladle quantities required above must be

implemented by the students.

Task 2: Bayesian mixture modelling with MCMC

The purpose of this task is to investigate Bayesian computation in a model with latent variables, multimodality, and an identifiability issue.

Consider the two-component Gaussian mixture model

$$Y_i \mid Z_i = k, \mu_1, \mu_2, \sigma^2 \sim N(\mu_k, \sigma^2), \quad k \in \{1, 2\},$$

where

$$P(Z_i = 1) = \pi, \quad P(Z_i = 2) = 1 - \pi.$$

Equivalently, after integrating out the latent allocation Z_i ,

$$p(y_i \mid \mu_1, \mu_2, \sigma^2, \pi) = \pi \varphi(y_i; \mu_1, \sigma^2) + (1 - \pi) \varphi(y_i; \mu_2, \sigma^2),$$

where $\varphi(\cdot; \mu, \sigma^2)$ denotes the normal density.

Use the prior specification

$$\mu_1, \mu_2 \stackrel{\text{ind}}{\sim} N(m_0, s_0^2), \quad \pi \sim \text{Beta}(a, b), \quad \sigma^2 \sim \text{InvGamma}(\alpha_0, \beta_0).$$

Choose suitable hyperparameter values and justify your choices. You may standardize the data when this makes the prior specification easier to interpret.

Part A: Latent-variable representation and Gibbs sampler

Introduce latent allocation variables $Z_1, \dots, Z_n$.

Derive the full conditional distributions required for a Gibbs sampler for

$$Z_1, \dots, Z_n, \quad \mu_1, \quad \mu_2, \quad \pi, \quad \sigma^2.$$

Implement the Gibbs sampler yourself in R. The main reported analysis must use your own implementation rather than a ready-made mixture-model or MCMC routine.

Your implementation should store the quantities needed for posterior inference and for assessing the behaviour of the Markov chain. Explain how you initialize the sampler and how you determine the number of iterations to use.

Part B: Simulation study and computational behaviour

Use simulated data to validate your implementation.

Consider at least two situations:

1. a mixture whose two components are clearly separated;
2. a mixture whose two components overlap substantially.

Choose the parameters yourself and explain why the two settings differ in difficulty.

For each setting, investigate whether your sampler recovers the data-generating mixture and assess its computational behaviour. Use appropriate MCMC diagnostics and graphical summaries. In particular, investigate how the amount of overlap affects inference about the component parameters and the latent allocations.

Run several chains from meaningfully different initial values.

Part C: Label switching and identifiability

The likelihood of the mixture model is unchanged if the labels of the two components are exchanged:

$$(\mu_1, \mu_2, \pi) \longleftrightarrow (\mu_2, \mu_1, 1 - \pi).$$

Investigate the consequences of this symmetry for your MCMC output.

Your report should address the following questions:

- What does label switching look like in the simulated chains?
- Why can ordinary posterior summaries of μ_1 , μ_2 , and π be misleading when labels switch?
- Which posterior quantities remain meaningful without assigning fixed labels to the components?
- How can an identifiability restriction such as $\mu_1 < \mu_2$, or an appropriate post-processing strategy, be used to obtain interpretable component-specific summaries?

Do not simply impose an ordering from the beginning without discussing the underlying problem. The aim of this part is to demonstrate that you understand why label switching occurs and what it means for Bayesian inference.

Part D: Application to real data

Apply your Bayesian mixture model to the `eruptions` variable in R's built-in `faithful` data set:

```
data(faithful)
y <- faithful$eruptions
```

The variable records eruption durations of the Old Faithful geyser.

Use your sampler to investigate whether a two-component Gaussian mixture provides a useful description of the distribution of eruption durations.

Your analysis should include:

- posterior summaries for the mixture parameters, taking the label-switching issue into account;
- a graphical comparison of the observed data with the fitted mixture distribution;
- posterior uncertainty about the allocation of observations to the two components;

- an assessment of MCMC performance;
- a discussion of whether the two-component Gaussian mixture is an adequate description of these data.

Interpret the components cautiously. A fitted mixture can provide a useful statistical description without implying that the observations necessarily arise from two physically distinct populations.

Reporting requirements

The report should contain a concise theoretical description of the Bayesian mixture model, the derivation of the Gibbs sampler, an explanation of the implementation, the simulation study, the investigation of label switching, and the real-data analysis.

The choices made during the analysis should be justified. MCMC output should not be presented only as raw trace plots or software output: diagnostics and posterior summaries should be interpreted in terms of the statistical problem.

Ready-made Bayesian software may be used to check selected results, but the Gibbs sampler and the main analysis must be implemented by the students.

Suggested report structure and marking scheme

Task 1: Time series bootstrap and signal dimension — 15 points

1. Problem statement, theory and methodology: 4 points.
2. Implementation and validation: 5 points.
3. Simulation study and analysis: 4 points.
4. Application to `timeseries.dat` and conclusions: 2 points.

Task 2: Bayesian mixture modelling with MCMC — 15 points

1. Bayesian model, latent-variable representation and full conditionals: 4 points.
2. Gibbs sampler implementation and computational methodology: 4 points.
3. Simulation study, diagnostics and label switching: 3 points.
4. Application to the `faithful` data and posterior interpretation: 3 points.
5. Conclusions: 1 point.

Total: 30 points.